

New User Flow — Organization-Based Architecture

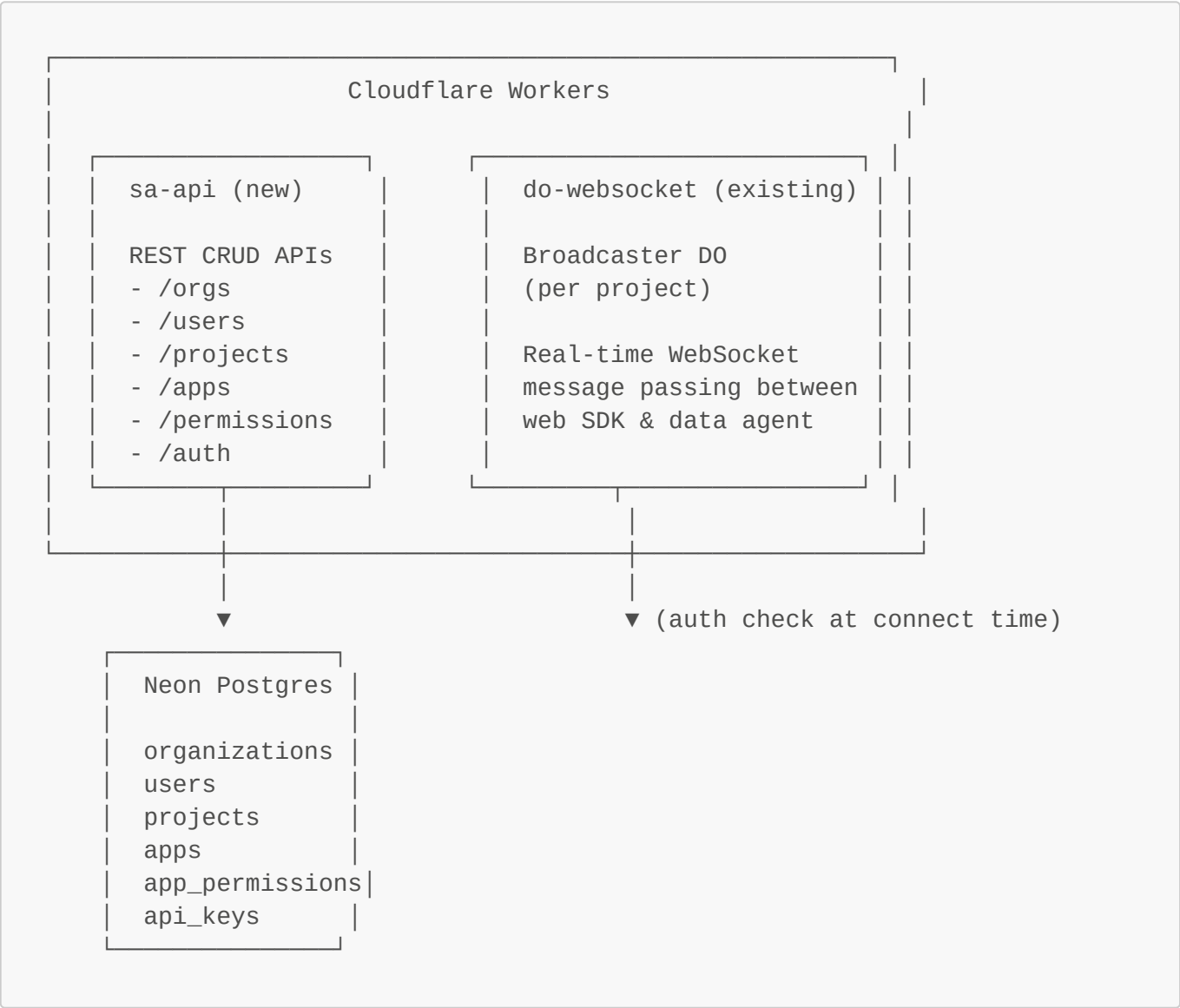
Overview

Moving from a **project-centric** user model to an **organization-centric** model.

Before: Users lived inside a project. Each project was an isolated silo.

After: Users belong to an organization. Org admin creates apps (dashboards, reports, chat agents, etc.) and grants individual app access to users. Users can ONLY see and login to apps they've been given permission to.

Architecture



Two workers, one database:

Worker	Purpose
sa-api (new)	All REST CRUD — org management, user management, project/app CRUD, permissions

Worker	Purpose
do-websocket (existing)	Real-time WebSocket message passing. Only reads from DB to validate auth at connection time

Core Concept



What each user sees when they login:

- **alice** (org_admin): All apps across all projects
- **bob**: Sales Dashboard (view), Revenue Report (view)
- **carol**: Sales Dashboard (edit), Support Agent (edit), Campaign Dashboard (view)

Users don't "join projects". They get access to **individual apps**. Projects are just a grouping mechanism for the admin.

Database Schema (Neon Postgres)

organizations

Column	Type	Description
id	UUID (PK)	Auto-generated
name	VARCHAR(255)	Organization display name
slug	VARCHAR(100) UNIQUE	URL-friendly identifier
created_at	TIMESTAMP	Default NOW()
updated_at	TIMESTAMP	Default NOW()

users

Column	Type	Description
id	UUID (PK)	Auto-generated
org_id	UUID (FK → organizations)	The organization this user belongs to
email	VARCHAR(255) UNIQUE	User email
name	VARCHAR(255)	Display name
role	ENUM('org_admin', 'member')	Organization-level role
is_active	BOOLEAN	Default true
created_at	TIMESTAMP	Default NOW()
updated_at	TIMESTAMP	Default NOW()

org_admin — Full access to everything in the org. Can create projects, apps, manage users, assign permissions. **member** — Can ONLY access apps explicitly granted by admin. Sees nothing else.

projects

Column	Type	Description
id	UUID (PK)	Auto-generated
org_id	UUID (FK → organizations)	Parent organization
name	VARCHAR(255)	Project name
slug	VARCHAR(100)	URL-friendly name (unique within org)
created_by	UUID (FK → users)	User who created it
created_at	TIMESTAMP	Default NOW()
updated_at	TIMESTAMP	Default NOW()

UNIQUE constraint on (org_id, slug).

Projects are an **admin-only grouping** concept. Regular users don't interact with projects directly — they only see their permitted apps.

apps

Column	Type	Description
id	UUID (PK)	Auto-generated
project_id	UUID (FK → projects)	Parent project
type	ENUM('dashboard', 'app', 'report', 'chat_agent')	App type
name	VARCHAR(255)	App display name

Column	Type	Description
config	JSONB	App-specific configuration
created_by	UUID (FK → users)	User who created it
is_active	BOOLEAN	Default true
created_at	TIMESTAMP	Default NOW()
updated_at	TIMESTAMP	Default NOW()

app_permissions (THE access control layer)

Column	Type	Description
id	UUID (PK)	Auto-generated
user_id	UUID (FK → users)	
app_id	UUID (FK → apps)	
permission	ENUM('view', 'edit', 'admin')	What the user can do
granted_by	UUID (FK → users)	Who granted this permission
created_at	TIMESTAMP	Default NOW()

UNIQUE constraint on (user_id, app_id).

This is the **single table that controls what a user can access**. No entry here = no access.

api_keys (existing table, updated)

Column	Type	Description
id	SERIAL (PK)	Auto-generated
project_id	VARCHAR(255)	Linked to project
org_id	UUID (FK → organizations)	New: linked to org
key_hash	VARCHAR(64)	SHA-256 hash
key_prefix	VARCHAR(12)	First 12 chars for identification
is_active	BOOLEAN	Default true
created_by	UUID (FK → users)	Who created it
description	TEXT	Optional
created_at	TIMESTAMP	Default NOW()
last_used_at	TIMESTAMP	Nullable

API Endpoints (sa-api worker)

Auth

Method	Endpoint	Description
POST	/auth/login	Login with email/password, returns JWT
POST	/auth/logout	Invalidate session
GET	/auth/me	Get current user + org + list of permitted apps

Organizations (org_admin only)

Method	Endpoint	Description
POST	/orgs	Create organization
GET	/orgs/:orgId	Get organization details
PUT	/orgs/:orgId	Update organization

Users (org_admin only)

Method	Endpoint	Description
POST	/orgs/:orgId/users	Create/invite user to org
GET	/orgs/:orgId/users	List all users in org
GET	/orgs/:orgId/users/:userId	Get user details + their app permissions
PUT	/orgs/:orgId/users/:userId	Update user (role, name, active status)
DELETE	/orgs/:orgId/users/:userId	Deactivate user (revokes all app access)

Projects (org_admin only)

Method	Endpoint	Description
POST	/orgs/:orgId/projects	Create project
GET	/orgs/:orgId/projects	List all projects in org
GET	/orgs/:orgId/projects/:projectId	Get project details + its apps
PUT	/orgs/:orgId/projects/:projectId	Update project
DELETE	/orgs/:orgId/projects/:projectId	Delete project (cascades to apps + permissions)

Apps (org_admin only)

Method	Endpoint	Description
POST	/projects/:projectId/apps	Create app in project
GET	/projects/:projectId/apps	List all apps in project

Method	Endpoint	Description
GET	/apps/:appId	Get app details
PUT	/apps/:appId	Update app
DELETE	/apps/:appId	Delete app (cascades permissions)

App Permissions (org_admin only)

Method	Endpoint	Description
POST	/apps/:appId/permissions	Grant user access to app
GET	/apps/:appId/permissions	List who has access to this app
PUT	/apps/:appId/permissions/:userId	Update user's permission level
DELETE	/apps/:appId/permissions/:userId	Revoke user's access to app

User-Facing (authenticated member)

Method	Endpoint	Description
GET	/my/apps	List all apps the logged-in user can access
GET	/my/apps/:appId	Get app details (only if user has permission)

User Flows

Flow 1: Org Admin Sets Up Everything

```
1. Admin creates org
  POST /orgs { name: "Acme Corp", slug: "acme" }

2. Admin creates users in the org
  POST /orgs/:orgId/users { email: "bob@acme.com", name: "Bob", role:
"member" }
  POST /orgs/:orgId/users { email: "carol@acme.com", name: "Carol", role:
"member" }

3. Admin creates a project (just a container for apps)
  POST /orgs/:orgId/projects { name: "Analytics", slug: "analytics" }

4. Admin creates apps inside the project
  POST /projects/:projectId/apps { name: "Sales Dashboard", type:
"dashboard" }
  POST /projects/:projectId/apps { name: "Support Agent", type:
"chat_agent" }

5. Admin grants app access to specific users
  POST /apps/:salesDashboardId/permissions { userId: bob_id, permission:
```

```
"view" }
  POST /apps/:salesDashboardId/permissions { userId: carol_id, permission:
"edit" }
  POST /apps/:supportAgentId/permissions { userId: carol_id, permission:
"edit" }
```

Flow 2: Member Logs In and Sees Their Apps

1. User logs in


```
POST /auth/login { email: "bob@acme.com", password: "..."}
→ Returns JWT with { userId, orgId, role: "member" }
```
2. Frontend fetches user's permitted apps


```
GET /my/apps (with JWT in Authorization header)
```

Backend query:

```
SELECT a.id, a.name, a.type, p.name as project_name, ap.permission
FROM app_permissions ap
JOIN apps a ON a.id = ap.app_id
JOIN projects p ON p.id = a.project_id
WHERE ap.user_id = $1 AND a.is_active = true
```

→ Returns:

```
[
  { id: "...", name: "Sales Dashboard", type: "dashboard", permission:
"view" }
]
```
3. User clicks on "Sales Dashboard"
 - Frontend loads the dashboard app
 - If the app needs real-time data, opens WebSocket (see Flow 3)

Flow 3: User Opens an App That Uses WebSocket (Real-Time)

1. User has already logged in and has JWT
2. User opens an app (e.g., a chat agent or live dashboard)

Frontend first validates app access:

```
GET /my/apps/:appId → returns app details + projectId
```
3. Frontend opens WebSocket to do-websocket worker


```
ws://do-websocket.../websocket?
projectId=xxx&appId=yyy&token=JWT&type=runtime
```
4. do-websocket worker validates:
 - Decode JWT → extract userId, orgId
 - Query Neon: "Does this user have permission for this app?"


```
SELECT 1 FROM app_permissions ap
JOIN apps a ON a.id = ap.app_id
```

```

    JOIN projects p ON p.id = a.project_id
    WHERE ap.user_id = $1 AND ap.app_id = $2 AND p.org_id = $3
  - If NO → reject with 403
  - If YES → route to Broadcaster D0 (keyed by projectId)

```

5. Broadcaster D0 handles real-time messaging as usual (runtime ↔ data-agent ↔ admin)

Flow 4: Data Agent (Node.js SDK) Connects

1. Data agent uses an API key (existing flow, unchanged)
`ws://do-websocket.../websocket?projectId=xxx&apiKey=sa_live_xxx&type=data-agent`
2. do-websocket validates API key against Neon (existing logic)
3. Routes to Broadcaster D0 for real-time messaging

Flow 5: Admin Revokes a User's Access

1. Admin removes user's app permission
`DELETE /apps/:appId/permissions/:userId`
2. Next time the user:
 - Calls `GET /my/apps` → that app no longer appears
 - Tries to open WebSocket for that app → rejected with 403
 - Active WebSocket connections continue until they disconnect (optionally: force-close via admin action)

Permission Model

```

org_admin
  → Implicit access to ALL projects and ALL apps in the org
  → Can create/delete projects, apps, users
  → Can grant/revoke permissions

member
  → Sees ONLY apps listed in app_permissions for their user_id
  → Cannot see projects directly (projects are admin grouping)
  → Permission levels per app:
    view → read-only access to the app
    edit → can modify data/content within the app
    admin → can configure the app (but NOT manage other users)

```

What Changes in do-websocket Worker

Minimal changes to the existing worker:

1. Accept **token** (JWT) and **appId** query params alongside existing **apiKey**
2. If **token** is provided:
 - Decode and verify the JWT
 - Query **app_permissions** to confirm user has access to the app
 - Pass **userId**, **orgId**, and **appId** into Broadcaster DO as WebSocket metadata
3. If **apiKey** is provided: existing flow (unchanged)

Everything else (Broadcaster DO, message routing, ping/pong, hibernation) stays the same.